Laboratorio 2.1: Dominio del Ciclo de Vida y Persistencia de Estado

1. Contexto y Objetivos

En Android 16 (API 36), el sistema operativo es más agresivo con la gestión de recursos para maximizar la eficiencia energética. Como se estudia en la teoría de la Semana 4, un Activity es un componente efímero que puede ser destruido por el sistema en cualquier momento (rotación de pantalla, falta de memoria).

Objetivos:

- **Experimentar** los callbacks del ciclo de vida mediante trazabilidad en Logcat.
- **Implementar** un ViewModel para garantizar la supervivencia de datos ante cambios de configuración.
- **Dominar** el uso de SavedStateHandle para restaurar el estado tras la muerte del proceso (Process Death).

2. Requisitos Previos

- Haber completado el **Laboratorio 1** (Estandarización del Entorno).
- Contar con el emulador **Pixel 8 - API 36** (Android 16).
- Tener habilitadas las **Opciones de Desarrollador** en el emulador.

3. Descripción General

El estudiante desarrollará una aplicación de contador que servirá como "sujeto de pruebas". A través de cuatro fases, observará cómo los datos se pierden y cómo las herramientas modernas de arquitectura (Jetpack) resuelven estos problemas de forma transparente para el usuario.

---

4. Paso a Paso Detallado

Fase 0: Preparación del Proyecto (Continuidad Técnica)

1. En el explorador de archivos de su sistema, **copie** la carpeta del Laboratorio 1.
2. **Pegue** la carpeta y renómbrela como RDA1_Lab6_Lifecycle_NombreApellido.
3. **Borre** las carpetas ocultas .gradle y build para limpiar la caché.
4. Abra **Android Studio Ladybug**, vaya a **File > Open** y seleccione la nueva carpeta.

5. Navegue a **Gradle Scripts > settings.gradle.kts** y cambie el nombre: rootProject.name = "LifecycleMaster".
6. Haga clic en el icono del **Elefante (Sync Project with Gradle Files)**.

Fase 1: Rastreo del Ciclo de Vida (Logging)

1. Abra el archivo app/src/main/java/[tu.paquete]/MainActivity.kt.
2. **Reemplace** la clase MainActivity con el siguiente código para registrar los callbacks:

```
class MainActivity : ComponentActivity() {
    private val TAG = "LIFECYCLE_DEBUG"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate: La Activity se está creando")
        // Aquí se infla la UI (Paso siguiente)
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart: La Activity es visible")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume: La Activity tiene el foco (interactuable)")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause: La Activity pierde el foco")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop: La Activity ya no es visible")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: La Activity va a ser destruida")
    }
}
```

Fase 2: El Problema (Pérdida de Datos en Rotación)

1. Dentro del onCreate, agregue un contador local simple.
2. **Sustituya** el bloque setContent por una UI básica:

```
setContent {
    var localCounter by remember { mutableStateOf(0) } // Variable local efímera
```

```
    Column(modifier = Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally) {
        Text(text = "Contador Local: $localCounter", style = MaterialTheme.typography.headlineLarge)
        Button(onClick = { localCounter++ }) {
            Text("Incrementar")
        }
    }
}
```

3. **Ejecución:** Inicie la App. Incremente el contador a 10.
4. **Prueba de Rotación:** Presione Ctrl + F12 en el emulador para rotar la pantalla.
5. **Observación:** El contador volverá a 0. Revise el **Logcat** filtrando por "LIFECYCLE_DEBUG" y verá que se llamó a onDestroy y luego a onCreate.

Fase 3: La Solución ViewModel (Supervivencia a Rotación)

1. Haga clic derecho en su paquete -> **New > Kotlin Class/File**. Nombre: CounterViewModel.
2. **Agregue** el siguiente código:

```
class CounterViewModel(private val savedStateHandle: SavedStateHandle) : ViewModel() {
    // Usamos SavedStateHandle para que el valor sobreviva incluso a Process Death
    private val _count = savedStateHandle.getMutableStateFlow("counter_key", 0)
    val count = _count.asStateFlow()

    fun increment() {
        _count.value++
    }
}
```

3. Vuelva a MainActivity.kt y **reemplace** la lógica del setContent para usar el ViewModel:

```
// Importante: Agregar esta dependencia en el onCreate
val viewModel: CounterViewModel = viewModel()
val count by viewModel.count.collectAsState()

setContent {
    Column(...) {
        Text(text = "Contador Seguro: $count")
        Button(onClick = { viewModel.increment() }) { Text("Incrementar") }
    }
}
```

Fase 4: Prueba (Process Death)

1. En el emulador, vaya a **Settings > System > Developer Options**.
2. Active la opción **"Don't keep activities"** (No conservar actividades).
3. Vuelva a su App, incremente el contador a 5.
4. Presione el botón **Home** (Círculo) para enviar la App al background. Android destruirá la Activity inmediatamente.
5. Vuelva a abrir la App desde el menú de aplicaciones recientes.
6. **Verificación:** Gracias al SavedStateHandle en el ViewModel, el valor 5 se mantendrá. Si solo hubiera usado una variable normal en el ViewModel sin SavedStateHandle, el valor se habría perdido.

5. Resultados Esperados

- El **Logcat** debe mostrar la secuencia completa de callbacks al rotar (onPause -> onStop -> onDestroy -> onCreate).
- La aplicación **no debe perder el valor** del contador al rotar la pantalla.
- La aplicación **debe restaurar el valor** tras simular la muerte del proceso con la opción de desarrollador activada.

6. Desafío: Trazabilidad de Inicio

Reto: Android 16 introduce ApplicationStartInfo. Modifique el onCreate para imprimir en los logs si la aplicación se inició de forma "Fría" (Cold Start) o "Cálida" (Warm Start) usando los conceptos de la guía docente.

7. Entregables

1. Informe **PDF** con capturas de pantalla de:
    - Logcat mostrando el ciclo de vida completo durante una rotación.
    - El emulador con el contador en 15 tras haber rotado la pantalla.
    - Una foto o captura del interruptor "Don't keep activities" activado en su emulador.
    - Copia del código como Anexo
    - Respuestas a preguntas
    - Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc
2. **Enlace al repositorio de GitHub** con el commit final: Lab 2.1: Persistencia total con ViewModel y SavedStateHandle.

8. Criterios de Evaluación

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe